

ADTA 5900/5560: Assignments

Important Recommendations

Thuan L Nguyen, PhD

1. Overview

To build, train, and test neural networks, the student needs to write Python code in Jupyter Notebook documents. In addition to writing the code, the student also needs to **add extra text** (using Markdown) to make **the code more manageable, legible, understandable, also easier to learn, check, and avoid mistakes.**

Also, **training and evaluating neural networks take time.** It may take **hours, even days or weeks,** to train a very deep neural network on a big data set. If the student's computer gets into the **hibernation** or **sleep state** while the training occurs, the **connection** to the remote deep learning server would be **disconnected.** Therefore, the student **should disable the computer hibernation or sleep feature** before starting training or evaluating a neural network.

2. Recommendation of Coding 1: Code in Sections

The **code** should be grouped in **sections**:

- Each **section** may consist of **only one** Jupyter Notebook cell or **multiple** cells.
- Each Jupyter Notebook **cell** may contain **only one** line of code or **multiple** lines of code.

Each **section** of code is done to complete **a step** that represents **one small important task** that the student **has to perform** in the process of building, training, and evaluating the neural network.

3. Recommendation of Coding 2: Clearly-Labeled Sections of Code

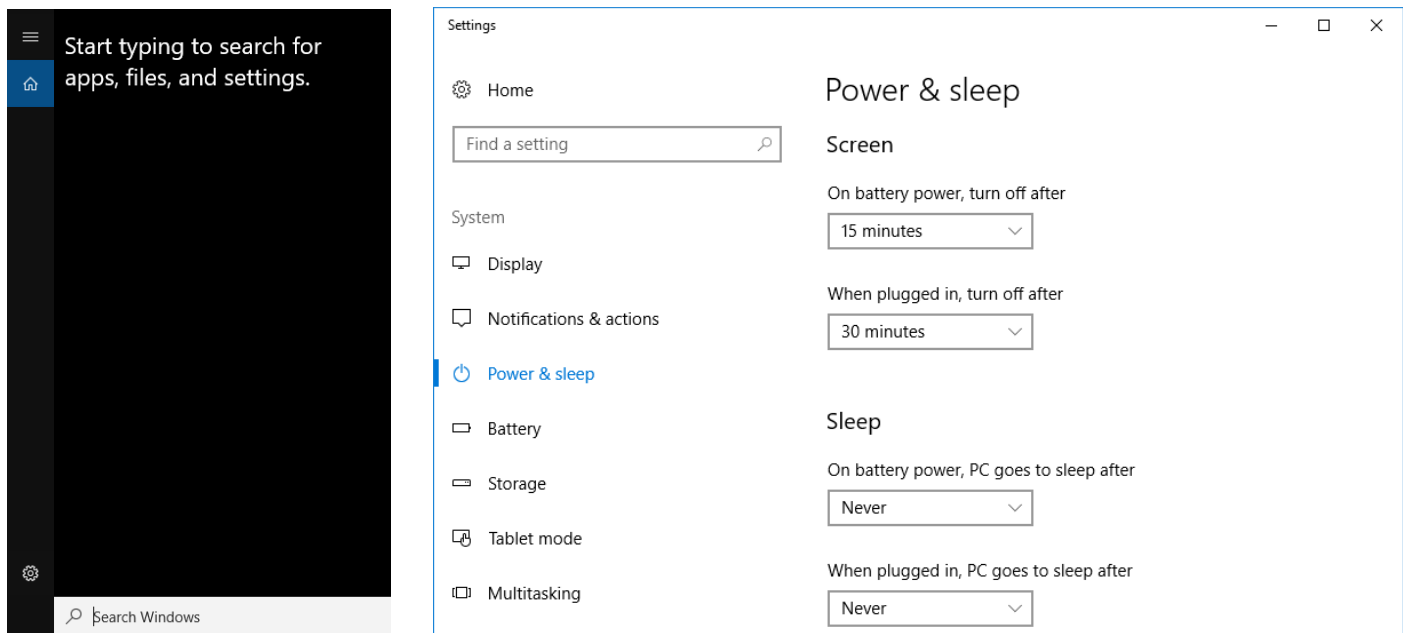
Each **section** of code should be **clearly labeled** by **text** in **one cell – using Markdown** – right over the Jupyter Notebook cell of the section. The text should **describe the task** that the student aims to perform with the section of the code.

The **text label** should be **accurate** and **brief**.

4. Recommendation of Disabling Computer Hibernation/Sleep Feature

For **Windows 10**:

- > Access **Cortana**
- > Enter for Search Windows: **Power & Sleep settings** OR Power Management



- > Look for the “**Sleep**” feature
- > Select “**Never**” for all the options (differences in Desktop and Laptop)

(NOTES: *Student can enable the feature again after finishing the work*)

5. Examples of Code in Labeled Sections

5.1 Example 1: One Section of a Single Jupyter Notebook Cell

Initialize Weights in Filter

```
# This function returns a tf.Variable used to store weights in a filter  
# This variable is initialized with values that can be used to initialize weights  
# The values are random numbers  
  
def initialize_weights (filter_shape):  
  
    init_random_dist = tf.truncated_normal(filter_shape, stddev=0.1)  
  
    return ( tf.Variable(init_random_dist) )
```

Initialize Bias

```
# This function returns a tf.Variable used to store bias  
# This variable is initialized with values that can be used to initialize bias  
# The value is initialized to 0.1  
  
def initialize_bias (bias_shape):  
  
    initial_bias_vals = tf.constant (0.1, shape=bias_shape)  
  
    return ( tf.Variable (initial_bias_vals) )
```

5.2 Example 2: One Section of Multiple Jupyter Notebook Cells

Create Placeholders for Inputs and Labels: x & y_true

```
# PLACEHOLDER  
  
# Create a placeholder for the inputs data: x  
# x: a 2D array  
# x: a placeholder that can hold any number of rows/record  
# --) Each row/record is a vector (1D array) to hold data for one image  
# --) Each row/record/image has 784 values/elements: 1 pixel = 1 value  
  
x = tf.placeholder(tf.float32, shape=[None, 784])
```

```
# PLACEHOLDER  
  
# Create a placeholder for the labels of the inputs data: y_true  
# y_true: a 2D array  
# y_true: Can hold any number of rows/records  
# Each row/record is a vector (1D-array) of 10 values that indicate a digit between 0 .. 9  
# --) Each row is the label of one image, i.e. one digit  
# --) Each row/record: The label is stored in the one-hot format  
# --) One-hot format: [1 0 0 0 0 0 0 0 0] --> 0; [0 0 0 0 0 0 0 1 0] --> 8  
  
y_true = tf.placeholder(tf.float32, [None, 10])
```

6. Why Do We Need to Write Code in Labeled Sections?

1. Making the **code** more **manageable, legible, and understandable**
 - a. With labeled sections, it is easier to manage the code, especially when changes are required.
 - b. When the code is divided into sections with labels, it is easier to read and understand it, which is **very important – and helpful – for anyone**, no matter whether he/she has much experience in programming or not.
2. **Easier to learn**
 - a. By grouping the **code** in **sections and labeling them**, the student **clearly knows what he/she is doing with the code**. As a result, he/she knows the code better and learn the techniques faster.
3. **Easier to check**
 - a. By grouping the code in sections and labeling them, the student can easily check to find out which steps he/she has done and which one he/she might miss.
 - b. Otherwise, the student has to read each line of code to find out which steps he/she has done and which one he/she might miss, which would be very time-consuming and pretty hard for those who did not yet have much experience in programming.
4. **Easier to debug and fix errors**
 - a. When some errors occurred, with the code separated in labeled sections that are more legible and understandable, it would be easier to find out where the error occurred.